

Application Note – H-protocol

DOCUMENT APPROVAL

	Author	Reviewer	Approver
Name	David Hodgson	Mariano Barbagallo	
Role			
Date	27 th September 2022	27 th September 2022	
Comments			

DOCUMENT INFORMATION

Document Title	AppNote – H-protocol	
Document Reference		
Document Status	Draft	
Original Author	David Cartwright	
Revision	2.	
Issue Date	3 rd July 2023	
Archive Location		
Document Distribution	Customers / Authorised Distributors	
Applicable Products	Product List	Document applicable to
	PC Tools / DVT	
	DCDC 300W / 500W	
	Gen4 size 2 / 4 / 6	
	Gen4 size 8 / 10	Yes
	Gen4 DC	
	Dragon 8	Yes
	HVLP	Yes
	Gen5 HV DCDC	
	Gen5 size 9	Yes

REVISION HISTORY

Revision	Date	Details	Author
1.0	19 Oct 2015	First issue	D Cartwright
2.0	07 Mar 2021	Update to new template and update	D Hodgson
2.1	23 Aug 2021		D Hodgson
2.2	14 Oct 2021	Add checksum calculation	D Hodgson
2.3	27 Sept 2022	Correct HS3 message content and other improvements	D Hodgson
2.4	03 July 2023	Add details to HC timeout configuration	D Hodgson

CONTENTS

1	Introduction.....	5
1.1	Purpose and Scope.....	5
1.2	Related documents	5
1.3	Document Conventions	5
1.4	Glossary	5
1.5	Conditions for document usage	5
2	H-Protocol Definition	6
2.1	Introduction.....	6
2.2	J1939 Protocol Implementation and Configuration	6
2.2.1	General.....	6
2.2.2	Control Messages (Received by Controller)	7
2.2.3	Status and Debug Messages (Transmitted by Controller)	7
2.2.4	J1939 General Configuration	9
2.3	H-Protocol CAN Message Format.....	10
2.3.1	CAN Frame and Data Format	10
2.3.2	H-Protocol Control Message Types (HC1, HC2, HC3, HC4).....	11
2.3.3	H-Protocol Status Message Types (HS1, HS2, HS3, HS4, HS5).....	14
2.3.4	H-Protocol Debug Message Types (HD1, HD2, HD3, HD4, HD5)	16
2.3.5	H-Protocol Control Message Checks and Synchronisation	20
2.4	H-Protocol Configuration	21
3	H-Protocol State Machine Control	24
3.1	NMT Operational State	24
3.2	State Machine	24
3.2.1	Gen4 / HVLP / Dragon 8 state machine.....	24
3.2.2	Gen5 size 9 state machine.....	26
3.3	State Machine Implementation.....	26
3.4	System Status	27
3.4.1	HVLP / Dragon8 / Gen5	27
3.4.2	Gen4 size 8 / 10	28

4	H-Protocol Motor Control	29
4.1	Start-Up Procedure	29
4.2	Control Message Timeout	29
4.3	Direct Control of Id And Iq (Gen4 size 8 / 10 only)	30
4.4	Interface to Torque Control	30
4.4.1	Torque Limits and Available Torque	30
4.4.2	Inversion of Motor Forward Direction	32
5	H-Protocol to TracApp & PumpApp Integration	33
6	Differences between Gen4 H-Protocol and HVLP/D8/Gen5	34

FIGURES AND TABLES

Figure 1	– J1939 message routing	9
Figure 2	– CAN message ID	10
Figure 3	– H-protocol state machine (Gen4 / HVLP / Dragon8)	25
Figure 4	– H-protocol state machine (Gen5).....	26
Figure 5	– motor / regen switching.....	31

Table 1.	Related Documents	5
Table 2.	Typographical Conventions	5
Table 3.	Glossary	5
Table 4.	H-protocol message contents	10
Table 5.	HC1 message (Torque demands 1).....	11
Table 6.	HC2 message (Torque demands 2).....	11
Table 7.	HC3 message (Battery demands).....	11
Table 8.	control word.....	12
Table 9.	HC (control) data format	12
Table 10.	HC4 message (Direct current demands)	13
Table 11.	direct current control data format	13
Table 12.	HS1 message (Torque status 1).....	14
Table 13.	HS2 message (Torque status 2).....	14
Table 14.	HS3 message (Temperature status).....	14
Table 15.	HS4 message (Fault status)	15
Table 16.	HS5 message (Encoder status).....	15
Table 17.	status word	15
Table 18.	HS (status) data format	16
Table 19.	HD1 message (Currents debug)	17
Table 20.	HD2 message (Voltages debug).....	17
Table 21.	HD3 message (Temperatures A debug)	17
Table 22.	HD4 message (Temperatures B debug)	17
Table 23.	HD5 message (PMAC control debug).....	18
Table 24.	HD6 message (Induction motor control debug)	18
Table 25.	HD (debug) data format	19
Table 26.	H-protocol PGN.....	21
Table 27.	H-protocol PF	22
Table 28.	H-protocol configuration.....	22
Table 29.	J1939 state flags	23

Table 30. J1939 state flags (HVLP / Dragon 8)	27
Table 31. J1939 state flags (Gen4).....	28
Table 32. Gen4 PGNs.....	34

1 Introduction

1.1 Purpose and Scope

The purpose of this application note is to explain in simple terms what the H-Protocol is, how to use it, its form and configuration.

1.2 Related documents

Ref	Title	Revision
1	AppNote – Gen4 size8/10 H-protocol using CANalyzer	
2	AppNote – I-Protocol	
3	AppNote – HVLP+D8 Mapping J1939 Items to VehApp	
4	AppNote – HVLP+D8 Torque Conditioning	

Table 1. Related Documents

1.3 Document Conventions

Typeface	Indicates
 WARNING	Very important information for the document user. Use this to indicate instructions or constraints the document user must comply with to prevent serious problems.
 CAUTION	Important information for the document user. Use this to indicate instructions or constraints the document user must comply with to prevent undesirable problems.
 NOTE	Useful information for the document user.

Table 2. Typographical Conventions

1.4 Glossary

Term	Description

Table 3. Glossary

1.5 Conditions for document usage

Copyright © 2022 BorgWarner Inc. All rights reserved.

You are authorised to open and view any electronic copy we send you of this document within your organisation. Otherwise the material may not in whole or in part be copied, stored electronically or communicated to third parties without the prior written agreement of BorgWarner Inc.

2 H-Protocol Definition

2.1 Introduction

The H-Protocol provides a simple, fast and flexible way to control a motor controller's torque output. It is a **J1939 based Peer-to-Peer message protocol** to sequence and control the operation of BorgWarner motor controllers. The status messages are sent back to the controlling unit as J1939 broadcast messages.

As with all J1939 messages - data format for variable fields within the data payload is in Little Endian format (LSB lowest position in 8 byte CAN packet) and unused fields are filled with 0xFF (unavailable) and variables in the data fields should be all unsigned – **however** through evolution of the H-protocol some fields within the control messages and status messages contain signed numbers and therefore the protocol is not J1939 strictly compliant.

The controller itself runs both 11-bit CANopen protocols as well as 29-bit extended CAN J1939 based protocols on the same CAN bus. CANopen is the standard mechanism used for controller calibration and configuration and programming using the DVT tool. Although device configuration takes place using standard CANopen messages which are used to access the device's Object Dictionary, the proprietary H-Protocol is used to communicate motor control and status information over a CAN network.

H-Protocol is an enhanced motor control protocol. It is designed specifically for use with traction applications that require fast response times and a fine degree of control but is equally suited to any electronic motor control application.

H-Protocol allows a VCU to control:

- Inverter bridge state (through control word)
- Motor torque demand
- Motor speed limits in both the forward and reverse directions
- DC current limits for both drive and regeneration
- Overall torque limits for both driving and braking torques

H-Protocol allows a VCU to monitor:

- Motor speed and actual output torque
- Motor, inverter, and internal component temperatures
- DC voltages and currents
- Inverter bridge state (through status word)
- Fault codes

2.2 J1939 Protocol Implementation and Configuration

2.2.1 General

As described in the H-Protocol configuration, section 2.4, – the PGN messages associated with the H-Protocol can be configured to fit anywhere in the PDU1 PGN range for both data pages. This gives full flexibility from an OEM point of view. Note H-Protocol Transmit status PGN's can also be configured as Broadcast PDU2 types if required.

However, ideally the PGN range chosen should be limited to use proprietary PGN's to strictly adhere to the J1939 standard.

The following sections describe implementation of J1939 protocols with regards to both Received PGNs and Transmit PGNs for the HVLP / Dragon8 / Gen5 range of products.

2.2.2 Control Messages (Received by Controller)

For HVLP / Dragon8 / Gen5 products are concerned this is the only part of J1939 that is implemented so far as receive PGN messages are concerned.

The following J1939 receive messages are registered for the H-Protocol :-

- PGN_RX_TORQUE_DEMAND_1 termed **HC1**
- PGN_RX_TORQUE_DEMAND_2 termed **HC2**
- PGN_RX_CURRENT_LIMITS termed **HC3**
- PGN_RX_DIRECT_CURRENT_DEMANDS termed **HC4** (Gen4 size 8 / 10 only)

Each message type has timeout error checking associated with it – that is if a message type is not received within a configurable time out period the message is deemed invalid and is acted upon by the Protocol handler – see section 0 below concerning H-Protocol Message validity checks and synchronisation.

CANopen object 0x4655 sub-index 3 is a receive time out multiplier applied to each message type which in turn has its own period multiplier. The receive message timeouts are polled every 5ms and the timeout period associated with each message type is as follows in milliseconds :-

- PGN_RX_TORQUE_DEMAND_1 = **5ms** * 0x4655 sub-index 3
- PGN_RX_TORQUE_DEMAND_2 = **5ms** * 0x4655 sub-index 3
- PGN_RX_CURRENT_LIMITS = **50ms** * 0x4655 sub-index 3
- PGN_RX_DIRECT_CURRENT_DEMANDS = **5ms** * sub-index 3

Note the maximum configurable multiplier time (0x4655, 3) is 255 and the overall maximum timeout time is 1000ms (no matter what multiplier is configured).

2.2.3 Status and Debug Messages (Transmitted by Controller)

For Transmit PGN's a similar procedure occurs. Again, the PGN is registered for transmission, the rate at which the message is transmitted is configurable by CANopen object 0x4655 sub-index 4 (transmit period multiplier) for standard J1939 transmissions.

J1939 Debug messages are also registered for transmission and the rate for these transmissions is configurable via CANopen object 0x4655 sub-index 5.

Also, each message type has a multiplier period associated with it to determine the time between message type transmissions. The transmit function is polled every 5ms and when the accumulated transmit poll period exceeds the configured timeout period the associated message type is transmitted.

The following J1939 transmit messages are registered for the H-Protocol :-

- PGN_TX_TORQUE_INFO_1 termed **HS1**
- PGN_TX_TORQUE_INFO_2 termed **HS2**
- PGN_TX_TEMPERATURE_INFO termed **HS3**
- PGN_TX_FAULT_INFO termed **HS4**
- PGN_TX_ENCODER_INFO termed **HS5**

The transmit times for each message type is as follows in milliseconds :-

- PGN_TX_TORQUE_INFO_1 = 5ms * 0x4655 sub-index 4

-
- PGN_TX_TORQUE_INFO_2 = 5ms * 0x4655 sub-index 4
 - PGN_TX_TEMPERATURE_INFO = 5ms * 0x4655 sub-index 4
 - PGN_TX_FAULT_INFO = 100ms * 0x4655 sub-index 4
 - PGN_TX_ENCODER_INFO = 5ms * 0x4655 sub-index 4

As stated, Debug Messages are also registered for the H-Protocol and these message types are :-

- PGN_DEBUG_CURRENTS termed **HD1**
- PGN_DEBUG_VOLTAGES termed **HD2**
- PGN_DEBUG_TEMPERATURES_A termed **HD3**
- PGN_DEBUG_TEMPERATURES_B termed **HD4**

The following debug messages are build specific in that only one of the messages or none will be sent, and the data contained within is build specific.

- PGN_DEBUG_PMAC_CONTROL_DATA termed **HD5**
- PGN_DEBUG_FV_CONTROL_DATA termed **HD6**

The transmit times for each message type is as follows in milliseconds :-

- PGN_DEBUG_CURRENTS = 10ms * 0x4655 sub-index 5
- PGN_DEBUG_VOLTAGES = 10ms * 0x4655 sub-index 5
- PGN_DEBUG_TEMPERATURES_A = 1000ms * 0x4655 sub-index 5
- PGN_DEBUG_TEMPERATURES_B = 1000ms * 0x4655 sub-index 5
- PGN_DEBUG_FV_CONTROL_DATA = 10ms * 0x4655 sub-index 5

Note the maximum CANopen object configurable multiplier time is 1000ms (i.e. 1 second) for both standard H-Protocol J1939 messages and H-Protocol Debug messages.

To stop debug messages from being transmitted set the debug transmit multiplier (0x4655 sub-index 5) to 0.

The way J1939 messages are transmitted within the ECU is depicted in the drawing below :-

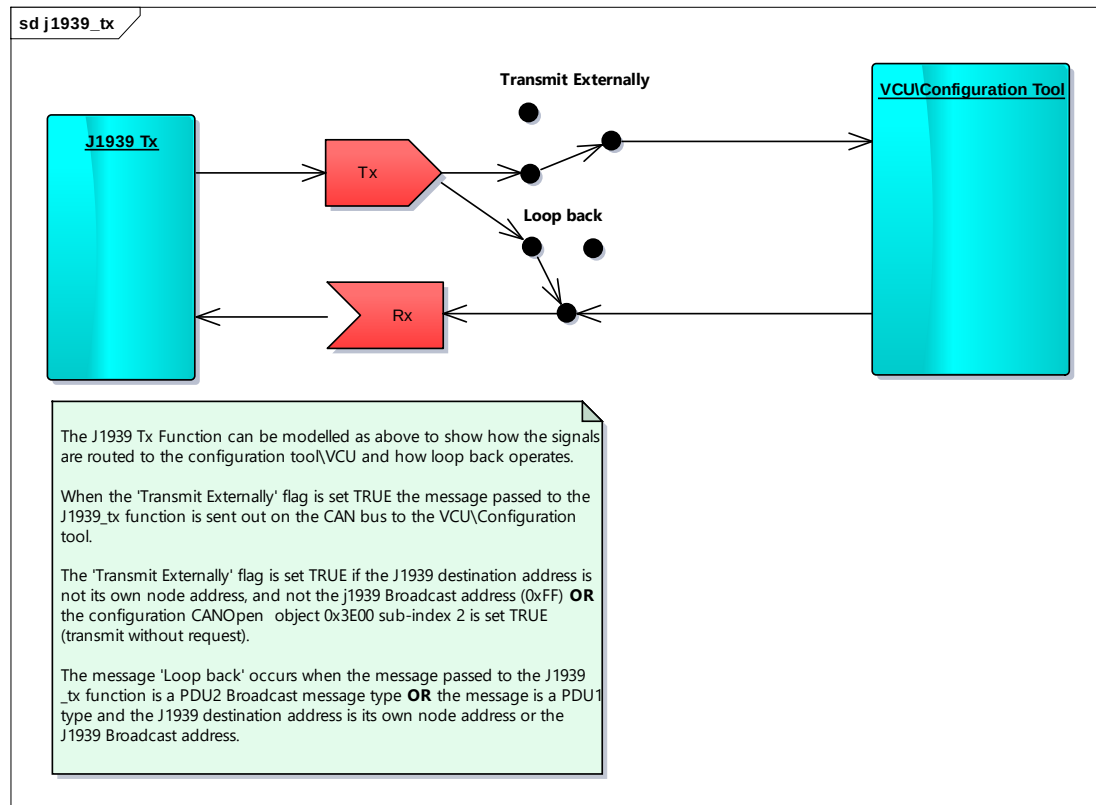


Figure 1 – J1939 message routing

So basically, depending on the H-Protocol configuration the messages can be simply routed out to the CAN (VCU or config tool) and/or looped back to the motor controller receive function handler. This technique allows features such as Traction Application to be utilised – discussed in section 5.

2.2.4 J1939 General Configuration

Two CANOpen object parameters set-up the operation of all J1939 protocols – these are object 0x3E00 sub-index 1 (J1939 source address – SA for the ECU) and sub-index 2 (transmit without request).

All other configurations are protocol specific and for the H-Protocol reference section 2.4.

2.3 H-Protocol CAN Message Format

2.3.1 CAN Frame and Data Format

The CAN 2.0B Frame is 29-bit extended format with the following fields :-

H-protocol message contents								
CAN Identifier	DB1	DB2	DB3	DB4	DB5	DB6	SEQ	CS

Table 4. H-protocol message contents

The data payload is 6 bytes, a sequence byte with an optional (configurable) checksum byte. The checksum check can be disabled for received messages by setting CANopen object 0x4655 sub index 2 to 0xE38E – any other value enables the check.

Key features of the data format are:

- Messages can be synchronised – “atomic” update.
- Integrity of the data can be ensured through the standard CAN CRC and acceptance mask check and the checksum of the payload and sequence count (SEQ) CS – Also included is an out of sequence check.
- The checksum calculation, if used, is the simple addition of data bytes DB1 through to DB6 and the ‘SEQ’ count
- Faulty CAN transmitters / failed control code may be detected.
- Suitable for use in some safety critical applications. (Depends on system safety level, and architecture)
- The BAUD rate for the ECU is configurable via the CANopen dictionary object 0x5900 sub-index 2 and supports 50k, 100k, 250k, 500k Baud and full speed CAN 2.0B 1M Baud

A sequence count is associated with each message type.

The receive sequence count is simply an 8-bit number that is incremented each time the controlling unit sends a new packet of that type – the count will eventually roll over and continue.

When the ECU sends a new status message the sequence count for that message type is incremented and placed in the message.

If the receiver gets a message whose sequence count difference ($\text{current_sequence_count} - \text{previous_sequence_count}$) is > 2 the message is deemed invalid and is acted upon by the Protocol handler – see section 0.

The CAN identifier's (message IDs) are designed to integrate with J1939 systems, consisting of a source address, PGN and priority fields. These fields are arranged as follows to give a 29-bit message identifier (note that R is reserved and is always set to zero):

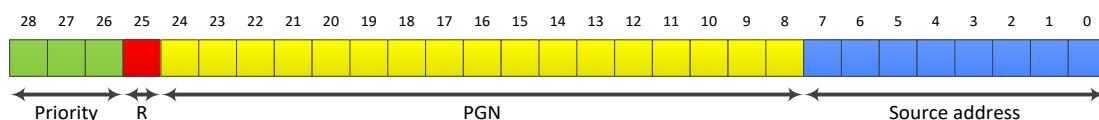


Figure 2 – CAN message ID

Note that the PGNs used by H-Protocol use PDU-1 format, which allows for peer-to-peer communication. The J1939 address of the controller can be configured at object 0x3E00 sub index 1, and any message generated by the controller will have this address encoded into the source address bits.

As these are PDU-1 format messages, the destination address will appear from bits 8 to 15. For H-Protocol status information generated by the controller, these are not directed at a specific node, and will be “broadcast” using a destination address of 0xFF. However, control information intended for the controller should be addressed to the controller explicitly, with the correct destination address.

2.3.2 H-Protocol Control Message Types (HC1, HC2, HC3, HC4)

As stipulated earlier there are 4 control message types associate with the H-Protocol, namely:

- PGN_RX_TORQUE_DEMAND_1 termed **HC1**
- PGN_RX_TORQUE_DEMAND_2 termed **HC2**
- PGN_RX_CURRENT_LIMITS termed **HC3**
- PGN_RX_DIRECT_CURRENT_DEMANDS termed **HC4** (Gen4 size 8/10 only)

The above messages are sent by the Vehicle Control Unit (VCU) and the message format for each type is, the PGN values indicated are the default values and are recommended.

Torque demand message 1 – Base period is 5ms (HC1)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11000	Torque demand		Control word		Drive torque limit		SEQ	CS
	Signed 16-bit		Bit field		Signed 16-bit			
	0.0625 Nm/bit		See Table 8		0.0625 Nm/bit			

Table 5. HC1 message (Torque demands 1)

Torque demand message 2 – Base period is 5ms (HC2)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11100	Regen torque limit		Forward speed limit		Reverse speed limit		SEQ	CS
	Signed 16-bit		Signed 16-bit		Signed 16-bit			
	0.0625 Nm/bit		1 RPM/bit		1 RPM/bit			

Table 6. HC2 message (Torque demands 2)

Battery demands – Base period is 50ms (HC3)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11200	Battery discharge current limit		Battery charge current limit		Target capacitor voltage		SEQ	CS
	Signed 16-bit		Signed 16-bit		Signed 16-bit			
	1 A/bit		1 A/bit		0.0625 V/bit			

Table 7. HC3 message (Battery demands)

Note: the SEQ (sequence) and CS (checksum) bytes which are detailed in section 0 of this document.

The above message's are supported by TracApp in HVLP and Dragon8 products when the PGN's are configured with the numbers above – see section 5 for more information.

The “Control word” in message **HC1** have bit fields used to give commands to, and check the state of, the H-Protocol state machine used to sequence to control of the motor – see section 3.

The values for control word are:

Value	Command	Description
0x0000	No command	Do nothing
0x0003	ENERGISE	Perform start up checks and execute pre-charge before entering the energised state with the bridge disabled
0x0005	ENABLE	If safe to pulse enter the enabled state with the bridge enabled
0x0006	SHUTDOWN	Enter the shutdown state with the bridge disabled
0x0009	FAULT RESET	If in the fault state due to an internal fault and reset command is received and no faults are active move to the initial NOT_READY_TO_SWICH_ON state – must recycle power to get out of fault state in existing H-Protocol implementation
0x000A	ACTIVE SHORT	Not used

Table 8. control word

The data size, engineering units and resolutions for the **HC1**, **HC2** and **HC3** message fields are listed in the table below:

	Signal	Format\Resolution	Minimum Limit	Maximum Limit
HC1	Controlword	Bit field	0	15
HC1	Torque Demand	Signed 16-bit, 0.0625 Nm/bit	-2048 Nm	+2047.9375 Nm
HC1	Drive Torque Limit	Signed 16-bit, 0.0625 Nm/bit	0 Nm	+2047.9375 Nm
HC2	Regen Torque Limit	Signed 16-bit, 0.0625 Nm/bit	-2048 Nm	0 Nm
HC2	Forward Speed Limit	Signed 16-bit, 1 RPM/bit	0 RPM	+32767 RPM
HC2	Reverse Speed Limit	Signed 16-bit, 1 RPM/bit	-32768 RPM	0 RPM
HC3	Battery Current discharge Limit (drive torque)	Signed 16-bit, 1 A/bit	0 A	+32767 A
HC3	Battery Current recharge Limit (regeneration braking)	Signed 16-bit, 1 A/bit	-32768 A	0 A
HC3	Target capacitor voltage (voltage control mode only)	Signed 16-bit, 0.0625 V/bit	-2048V	+2047.9375V

Table 9. HC (control) data format

On Gen4 only, the H-Torque protocol provides the ability to set Iq reference and Id reference directly from CANalyzer or a VCU with the **HC4** control message. This is useful for test purposes (motor characterisation), where a stable current demand is necessary. For example:

- Tuning of current control gains by applying step changes in demand
- Forcing a motor into field weakening before the normal saturation point
- Measuring torque output for a given motor current

The message format is as follows:

Direct Id\Iq Control demand message 1 – Base period is 5ms (HC4)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11300	Iq Direct Current (A)		Id Direct Current (A)		0xFF	0xFF	SEQ	CS
	Signed 16-bit		Signed 16-bit					
	0.0625 A/bit		0.0625 A/bit					

Table 10. HC4 message (Direct current demands)

In order to activate this object 0x4656, sub 0 must be set to 0x0103.

The message data type, units and resolution are as defined in the table below:

Signal	Format	Minimum limit	Maximum limit
Iq Current	Signed 16-bit, 0.0625 A/bit	-2048A*	+2047.9375A*
Id Current	Signed 16-bit, 0.0625 A/bit	-2048A*	+2047.9375A*

Table 11. direct current control data format

*Notes:

- Maximum currents for Id & Iq cannot exceed the control Is Max rating.
- Negative values for Id will be clamped if the system needs to go to field weakening (this will override what you set).
- Iq will be limited to Iq_max.
- Iq will also be limited to prevent the standard cutbacks being exceeded, e.g. speed limit, battery current limit, thermal cutback etc.

2.3.3 H-Protocol Status Message Types (HS1, HS2, HS3, HS4, HS5)

As stipulated earlier there are 5 status message types associated with the H-Protocol, namely:

- PGN_TX_TORQUE_INFO_1 termed **HS1**
- PGN_TX_TORQUE_INFO_2 termed **HS2**
- PGN_TX_TEMPERATURE_INFO termed **HS3**
- PGN_TX_FAULT_INFO termed **HS4**
- PGN_TX_ENCODER_INFO termed **HS5**

These messages are sent **only** when the motor controller is in operational mode. The message rate is configurable and the message format and type is as follows - note the PGN values indicated are the default values and are recommended:

Torque status message 1 – base period is 5ms (HS1)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11800	“Measured” Output Torque		Measured Motor Speed Note1		Battery (DC link) Current		SEQ	CS

Table 12. HS1 message (Torque status 1)

Note 1: In the HVLP/D8/Gen5 implementation of the H-Protocol a single pole filter is applied to the measured motor speed before being output – this filter is configurable and its time constant can be configured using CANopen object dictionary object 0x4655 sub-index 12.

Torque status message 2 – base period is 5ms (HS2)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11900	Available Forward Torque		Available Reverse Torque		Status word	Torque Limit code Note 2	SEQ	CS

Table 13. HS2 message (Torque status 2)

Note 2: where the torque limit code is:

- HVLP / Dragon 8 / Gen5 size 9
 - 0 = No limit is in effect
 - 1 = Max device power limit
 - 2 = Motor power limit map
 - 3 = Thermal cutback (motor or inverter)
 - 4 = Limits received via H-Protocol
 - 5 = IPM 3D lookup table calculated limit
 - 6 = Bridge is not enabled
 - 7 = DC (battery) voltage cutback map
 - 8 = DC (battery) current cutback
 - 9 = Speed limit

Temperature status message – base period is 5ms (HS3)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11A00	Inverter temperature remaining before cutback		Motor measured Temperature		Measured Capacitor Voltage		SEQ	CS

Table 14. HS3 message (Temperature status)

Fault status message – base period is 100ms (HS4)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11B00	Fault code		Debug bytes				SEQ	CS

Table 15. HS4 message (Fault status)

Encoder status message – base period is 5ms (HS5)								
Byte	0	1	2	3	4	5	6	7
PGN 0x11C00	Encoder Angle LSB	Encoder Angle ISB1	Encoder Angle ISB2	Encoder Angle MSB	0xFF	0xFF	SEQ	CS

Table 16. HS5 message (Encoder status)

The “Status word” in message **HS2** have fields used to give status information concerning the ECU controller –The lower 4 bits of Byte4 are used to identify the current state of the motor controller master state machine and bits 5 to 8 are currently unused. Byte5 in the H-Protocol currently contains the torque ‘limit code value’ – this identifies what the limiting factor is in the motor control output calculation. For details please refer to **AppNote - HVLP+D8 Torque Conditioning**.

The values for the motor controller state (lower 4 bits of Byte4) are indicated in the table below:

Value	State
0x0001	Not ready to switch on
0x0002	Shutdown
0x0004	Precharge
0x0007	Energised
0x0008	Enabled
0x000B	(*) Fault Reaction Active
0x000D	Fault off
0x000E	(*) Active Short

Table 17. status word

(*) Only used for some software builds

The size units and resolutions for the **HS1**, **HS2**, **HS3**, **HS4** and **HS5** message fields are listed in the table below:

	Signal	Format	Minimum Limit	Maximum Limit
HS1	“Measured” (estimated) Output Torque	Signed 16-bit, 0.0625 Nm/bit	-2048 Nm	2047.9375 Nm
HS1	Measured Motor Speed	Signed 16-bit, 1 RPM/bit	-32768 RPM	32767 RPM
HS1	Battery Current	Signed 16-bit,	-32768 A	32767 A

		1 A/bit		
HS2	Available Forward Torque	Signed 16-bit 0.0625 Nm/bit	-2048	2047.9375
HS2	Available Reverse Torque	Signed 16-bit, 0.0625 Nm/bit	-2048	2047.9375
HS2	Statusword	Decimal	0	15
HS2	Torque Limit Code	Decimal	0	255
HS3	Inverter temperature remaining before cutback	Signed 16-bit, 1 degC/bit	-32768 degC	32767 degC
HS3	Measured Motor Temperature	Signed 16-bit, 1 degC/bit	-32768 degC	32767 degC
HS3	Measured Capacitor Voltage	Signed 16-bit, 0.0625 V/bit	-2048	2047
HS4	(*1) Fault code	N/A	N/A	N/A
HS4	(*2) Debug bytes	Unsigned 32-bit	0	0
HS5	Encoder Angle	Unsigned 32-bit, $1.525 * 10^{(-5)}/\text{bit}$	0	1 pu

(*1) Fault_codes.xls

(*2) Debug bytes currently unused and set to 0

Table 18. HS (status) data format

2.3.4 H-Protocol Debug Message Types (HD1, HD2, HD3, HD4, HD5)

As stipulated earlier there are up to 5 debug message types (HD1->HD5) associated with the H-Protocol, namely:

- PGN_DEBUG_CURRENTS = 10 * 0x4655 sub-index 5 (**HD1**)
- PGN_DEBUG_VOLTAGES = 10 * 0x4655 sub-index 5 (**HD2**)
- PGN_DEBUG_TEMPERATURES_A = 1000 * 0x4655 sub-index 5 (**HD3**)
- PGN_DEBUG_TEMPERATURES_B = 1000 * 0x4655 sub-index 5 (**HD4**)
- *PGN_DEBUG_PMAC_CONTROL_DATA = 10 * 0x4655 sub-index 5 (**HD5**)
- *PGN_DEBUG_FV_CONTROL_DATA = 10 * 0x4655 sub-index 5 (**HD6 – not currently supported**)

***Note:** only HD5 is currently supported

As with all status message types the period of transmission is configurable via the object dictionary – see above. The messages themselves are normally for internal use to debug what is happening within the controller – the messages can be switched off by setting the debug multiplier time to 0 (0x4655 sub-index 5).

The format of each message type is as defined below – Note debug messages do not carry a sequence number and checksum – since for debug purposes and messages pack in as much information as possible - the PGN values indicated are the default values and are recommended:

Debug Currents (HD1) base period is 10ms

Byte	0	1	2	3	4	5	6	7
PGN 0x12800	Id Current actual		Iq Current actual		Id Current reference		Iq Current reference	

Table 19. HD1 message (Currents debug)

Debug Voltages (HD2) base period is 10ms								
Byte	0	1	2	3	4	5	6	7
PGN 0x12900	Ud Voltage		Uq Voltage		Modulation Index		Control Torque Target (Torque Actual)	

Table 20. HD2 message (Voltages debug)

Debug Temperatures A (HD3) base period is 1 second								
Byte	0	1	2	3	4	5	6	7
PGN 0x12A00	Junction 0	Junction 1	Junction 2	Junction 3	Junction 4	Junction 5	Motor Est	Motor Measured 1

Table 21. HD3 message (Temperatures A debug)

Debug Temperatures B (HD4) base period is 1 second								
Byte	0	1	2	3	4	5	6	7
PGN 0x12B00	Fast Cap	PCB / Bus- Bar	Slow Cap	Heatsink Note 1	Delta Junction heatsink	Thermal Cutback code Note 2	Motor measured 2 (Coolant)	Motor measured 3 (Winding)

Table 22. HD4 message (Temperatures B debug)

Note 1: Heatsink temperature is not applicable to Gen5 size 9

Note 2: where the temperature cutback code are as follows

- **HVLP**
 - 0x00, 0x01, 0x02, 0x03, 0x04, 0x05 IGBT switching junction temperature
 - 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B IGBT diode junction temperature
 - 0x0C, 0x0D, 0x0E, 0x0F, 0x10, 0x14, 0x16 PCB temperature
 - 0x11, 0x12, 0x18 Capacitor temperature
 - 0x13 Heatsink temperature
- **Dragon 8**
 - 0x00, 0x01, 0x02, 0x03, 0x04, 0x05 MOSFET junction temperature
 - 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B MOSFET junction delta temperature
 - 0x0C Battery switch temperature estimate
 - 0x0D, 0x0E, 0x0F, 0x10, 0x11, 0x12, 0x18 PCB Track temperature
 - 0x13, 0x14 Capacitor temperature
 - 0x15, 0x16, 0x17 Heatsink temperature
- **Gen5 size 9**
 - 0x00 to 0x05 IGBT junction temperature
 - 0x0C to 0x10 Busbar temperature
 - 0x11, 0x12, 0x18 Capacitor temperature

The following **HD5** and **HD6** messages are build dependant and only one can exist for any given build:

Debug PMAC Control (HD5) base period is 10ms								
Byte	0	1	2	3	4	5	6	7
PGN 0x12C00	Maximum Flux		Iv_Limit		Iq_Max		Ls (stator inductance)	

Table 23. HD5 message (PMAC control debug)

Debug Induction Vector Control (HD6) base period is 10ms								
Byte	0	1	2	3	4	5	6	7
PGN 0x12D00	Slip		Lm_Sat		Iq_Max		Recip_Flux	

Table 24. HD6 message (Induction motor control debug)

The size units and resolutions for the **HD1**, **HD2**, **HD3**, **HD4**, **HD5** and **HD6** message fields are listed in the table below:

	Signal	Format	Minimum Limit	Maximum Limit
HD1	Id actual current	Signed 16-bit, 0.0625 A/bit	-2048	2047.9375
HD1	Iq actual current	Signed 16-bit, 0.0625 A/bit	-2048	2047.9375
HD1	Id ref current	Signed 16-bit, 0.0625 A/bit	-2048	2047.9375
HD1	Iq ref current	Signed 16-bit, 0.0625 A/bit	-2048	2047.9375
HD2	Ud voltage	Signed 16-bit, 0.0625 V/bit	-2048	2047.9375
HD2	Uq voltage	Signed 16-bit, 0.0625 V/bit	-2048	2047.9375
HD2	Modulation Index	Unsigned 16-bit, 0.00305 %/bit	0	199.9969
HD2	Control Torque Target (Torque Actual)	Signed 16-bit, 0.0625 Nm/bit	-2048	2047.9375
HD3	Junction 0 temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD3	Junction 1 temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD3	Junction 2 temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD3	Junction 3 temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD3	Junction 4 temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD3	Junction 5 temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD3	Motor Est temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD3	Motor Measured 1 temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD4	Fast Capacitor temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD4	PCB / Busbar temperature	Unsigned 8-bit, 1 degC/bit	0	255

HD4	Slow Capacitor temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD4	Heatsink temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD4	Delta Tj temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD4	Temperature cutback code	No units	0	255
HD4	Motor Measured 2 (coolant) temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD4	Motor Measured 3 (winding) temperature	Unsigned 8-bit, 1 degC/bit	0	255
HD5	Maximum Flux	Signed 16-bit, 1 A/bit	-32768	32767
HD5	Iv_Limit	Signed 16-bit, A, in 1 A/bit	-32768	32767
HD5	Iq_Maximum	Signed 16-bit, A, in 1 A/bit	-32768	32767
HD5	Ls stator inductance	Unsigned 16-bit, 0.0596 uH/bit	0	3906 uH
HD6	Slip	Signed 16-bit, 0.0625 rad/s/bit	-2048	2047.9375
HD6	Lm_Saturated	Unsigned 16-bit, 0.15259 uH/bit	0	10018.9 uH
HD6	Iq_Maximum	Signed 16-bit, 0.0625 A/bit	-2048	2047.9375
HD6	Recip_Flux	Signed 16-bit, 0.0625 1/A/bit	-2048	2047.9375

Table 25. HD (debug) data format

2.3.5 H-Protocol Control Message Checks and Synchronisation

There are three mandatory control messages in the H-Protocol – they are **HC1**, **HC2** and **HC3**. In the software builds that support direct Id and Iq control, **HC4** is optional and enabled by setting CANOpen object 0x4656 sub-index 0 to 0x103. If **HC4** message is sent it becomes part of the validity and synchronisation checks but is not utilised in a control sense until it is enabled.

At start-up the motor controller starts in message timeout fault – once all mandatory messages, are sent at least **once** and all have been received without message timeouts the control inputs to the motor controller are updated and operated upon by the motor algorithm.

If incoming messages timeout all controller demands and limits are set to zero to prevent the motor from outputting torque – when **all** messages are received again without any timeouts the controller inputs are updated again and operated upon by the motor algorithm. Motor controller message timeout checks are performed every 5ms. The timeout periods for the control messages are detailed in section 2.2.2.

The responsiveness of the motor controller is determined by the motor set-up and the rate at which the controller is updated with control commands which is determined by the Vehicle Control Unit (VCU) control message update time and is typically 10ms.

In addition to the timeout checks validity checks are performed on each of the incoming messages. These checks consist of:

- a) Checking that the control message data length is 8 bytes long – if it is, proceed to check b) and if this check fails, move to check c)
- b) If it is the first time the message has been received the previous message sequence count is made equal to the current received message sequence count and a checksum check is performed on the data and sequence count and if it passes the message is deemed valid. At this point the message receive handler updates the required motor controller parameters and the receive message timeout time is reset to the timeout value indicated in section 2.2.2. If the message fails this check – the motor controller inputs and timeout value are not updated for that message until another valid message of that type has been received.
- c) If the message has been received more than once then a sequence message jump check is made to ensure that the current message sequence count minus the previous sequence count (sequence count difference) is less than 2 (if not it indicates two consecutive messages have been missed) and the message is invalid. If the message is deemed valid the message handler again updates the motor controller input parameters and the receive message timeout time is reset to the timeout value indicated in section 2.2.2– in addition the previous message sequence count is set to the current message sequence count – so the message is again synchronised.
- d) The checksum typically in the last byte is valid. It is simply calculated by summing the other 7 bytes of the message clamped to 0xFF

Note: If the check in c) fails then the message is not used and the controller inputs are not updated – in addition the message timeout is not reset and the previous message sequence count is not reset to the current message sequence count. This has the effect of freezing that controller input – until a message timeout occurs at which point the demand and limits to the motor controller are set to 0 to prevent output torque. Note this situation will last $253 \times \text{VCU update time}$ – assume a 10ms update the control of the motor could be lost for a considerable period of time whilst the incoming message sequence count rolls round until it matches the previous message sequence count.

Note if the checksum calculation is disabled using object 0x4655, 2 – then when messages are received, they are immediately deemed valid without any checks.

2.4 H-Protocol Configuration

As seen in section 2.2.4 there are two general parameters that set-up the J1939 communications – they are the ECU source address (SA) and the ‘transmit without request flag’ – these are set using CANopen object 0x3E00 sub-index 1 and sub-index 2.

The H-Protocol as applied to HVLP/Dragon8/Gen5 is a peer-peer protocol where all messages can be configured to lie anywhere in the PDU1 J1939 address range (PF value < 240). These can be configured via CANopen object 0x3F00 and is outlined in the table below – reference document in section 2.2.1 for J1939 definitions, however for convenience the PGN format for the 29-bit J1939 CAN ID is repeated below.

The CAN identifier of a J1939 message contains Parameter Group Number (PGN), source address, priority, data page bit, extended data page bit and a target address (only for a peer-to-peer).

The identifier is composed as follows:

Priority	Extended Data Page	Data Page	PDU Format (PF)	PDU Specific (PS)	Source Address
Bits 28.....26	Bit 25	Bit 24	Bits 23.....16	Bits 15.....8	Bits 7.....0
3 bit	1 bit	1 bit	8 bit	8 bit	8 bit

Table 26. H-protocol PGN

For peer-peer communication the PS value will be set to either the target ECU source address (destination address) or the peer-peer broadcast address 0xFF.

J1939 Function	Min	Max	Default	0x3F00 sub-index
J1939 Transmit Priority	0	7	6	1
J1939 Data Page	0	1	1	2
PF for HC1	0	0xEF	0x10	4
PF for HC2	0	0xEF	0x11	4
PF for HC3	0	0xEF	0x12	5
PF for HC4	0	0xEF	0x13	6
PF for HS1	0	0xEF	0x18	7
PF for HS2	0	0xEF	0x19	8
PF for HS3	0	0xEF	0x1A	9
PF for HS4	0	0xEF	0x1B	10
PF for HS5	0	0xEF	0x1C	11
PF for HD1	0	0xEF	0x28	12
PF for HD2	0	0xEF	0x29	13
PF for HD3	0	0xEF	0x2A	14

PF for HD4	0	0xEF	0x2B	15
PF for HD5	0	0xEF	0x2C	16
PF for HD6	0	0xEF	0x2D	17

Table 27. H-protocol PF

Object 0x4656, which is **only applicable to Gen4** is used to set the motor control mode of operation – the applicable values are:

- 0x0000 = Standard operation
- 0x0103 = Direct Id/Iq control (used if message **HC4** is used for motor control)
- 0x7050 = PMAC 2D torque accuracy tables
- 0x8060 = PMAC 3D IMPM tables

The remaining H-Protocol configuration is set via CANopen object 0x4655 as defined below:

Object	Function
0x4655,1	Allows you to invert the direction of forward rotation. Set to 0x5533 for normal operation. Setting this to 0xCCAA will invert the torque demands and speeds through the protocol so that positive torque demands will cause the motor to rotate in the opposite direction. IF unsure, set 0x5533.
0x4655,2	Set this to 0xE38E to disable the message validation checks – this is needed for CANalyzer. Setting this disables the check on the message counter and checksum flags.
0x4655,3	Timeout multiplier for received messages. The timeout for any message is the message's base rate (detailed below) multiplier by this value. If any message is not received after this time, the system outputs zero torque.
0x4655,4	Transmit message multiplier. This sets the rate at which outgoing messages are transmitted. Messages will be transmitted with a period of the message's base rate multiplied by this value.
0x4655,5	Transmit message multiplier but for debug messages. Setting to zero will disable transmission of debug messages.
0x4655,6	Status flags. These are read only.
(*1) 0x4655,7	If you want to use a line contactor, set this to the analogue output that the contactor is connected to. Set to zero if you don't want the controller to drive a line contactor – Gen4 only
(*1) 0x4655,8	Line contactor pull-in voltage – Gen4 only
(*1) 0x4655,9	Line contactor hold-in voltage – Gen4 only
(*1) 0x4655,10	Line contactor pull-in time – Gen4 only
(*1) 0x4655,11	Preferred J1939 source address. Used to calculate message IDs for status and control messages – Gen4 only
(*2) 0x4655,12	Motor speed feedback filter pole

(*1) Applicable to Gen4 only

(*2) Applicable to HVLP/D8/Gen5 only

Table 28. H-protocol configuration

Note the status flags (sub-index 6) are read only and the following bit flags have been currently been allocated for the H-Protocol:

Status Flag	Mask Value
STATUS_BRIDGE_CHARGED	0x0001
STATUS_ENCODER_INITIALISED	0x0002
STATUS_AUTOZERO_TAKEN	0x0004
STATUS_PULSE_INPUT_ENABLED	0x0008
STATUS_BOTTOM_SWITCHES_OK	0x0010
STATUS_TOP_SWITCHES_OK	0x0020
STATUS_PARAMS_RECIEVED	0x0040
STATUS_LOGIC_VOLTAGES_ESTABLISHED	0x0080
STATUS_PF_OF_KNOWN_TYPE	0x0100
STATUS_INPUTS_OK	0x0200
(*)STATUS_NO_SEVERE_FAULTS	0x0400
(*)STATUS_CAN_COMMS_OK	0x0800
STATUS_LINE_CONTACTOR_CLOSED	0x1000

Table 29. J1939 state flags

(*) HVLP / Dragon8 / Gen5 only return these two flags to object 0x4655 sub-index 6

3 H-Protocol State Machine Control

3.1 NMT Operational State

The NMT operational state machine operates as per the standard CANopen software. The pre-operational and operational states are available, and the node will switch between these on reception of the appropriate NMT control message.

The H-Protocol will not transmit any information unless it is in the operational state.

3.2 State Machine

In order to control operation of the bridge, a motor control state machine is defined. Various transitions are defined, gated on various conditions, such as the state of the hardware, measured signals, and commands received via the control word.

The overall principle of this state machine is to implement a simplified version of the CANopen CiA 402 state machine that exists in other software. Some of the state names are duplicated, and perform similar functions, but their purpose is simplified wherever possible.

In the **Power Enabled** super-state, the bridge is expected to be enabled and the current control PI loops are running and can be controlled by the VCU using the H-Protocol control commands. In **Power Disabled**, the bridge is expected to be disabled, and the current control PI loops are held in a reset state.

There should be no state where the bridge is enabled, but output current is not being controlled (i.e. pulsing at a fixed PWM of 50% is not supported). The only exception to this would be **Active Short Circuit** which, although currently not implemented, would represent a special mode of operation regarding bridge control, hence its isolation from both **Power Enabled** and **Power Disabled**.

The state machine is as depicted below with different implementations for Gen4, HVLP, Dragon8 and Gen5 size9.

Please note the commands to transition between states are given in the H-Protocol **HC1** message as detailed in section 2.3.2.

3.2.1 Gen4 / HVLP / Dragon 8 state machine

At present the implementation omits several parts of the state machine. The significant variations from the state machine in Figure 3 are: **Error! Reference source not found.** below are:

- The **Active Short Circuit** and **Fault Reaction Active** states are not currently used in H-protocol
- The notion of a “gravity 1” fault was never implemented. Instead, “gravity 2” refers to any level 3 or greater fault as per the existing fault handling scheme, which causes the bridge to disable.
- The fault recovery transition from **Fault Off** to **Not ready to switch on** is **only** implemented in the Gen4 J1939 build, in HVLP/Dragon8/Gen5 the power must be recycled to clear the fault.

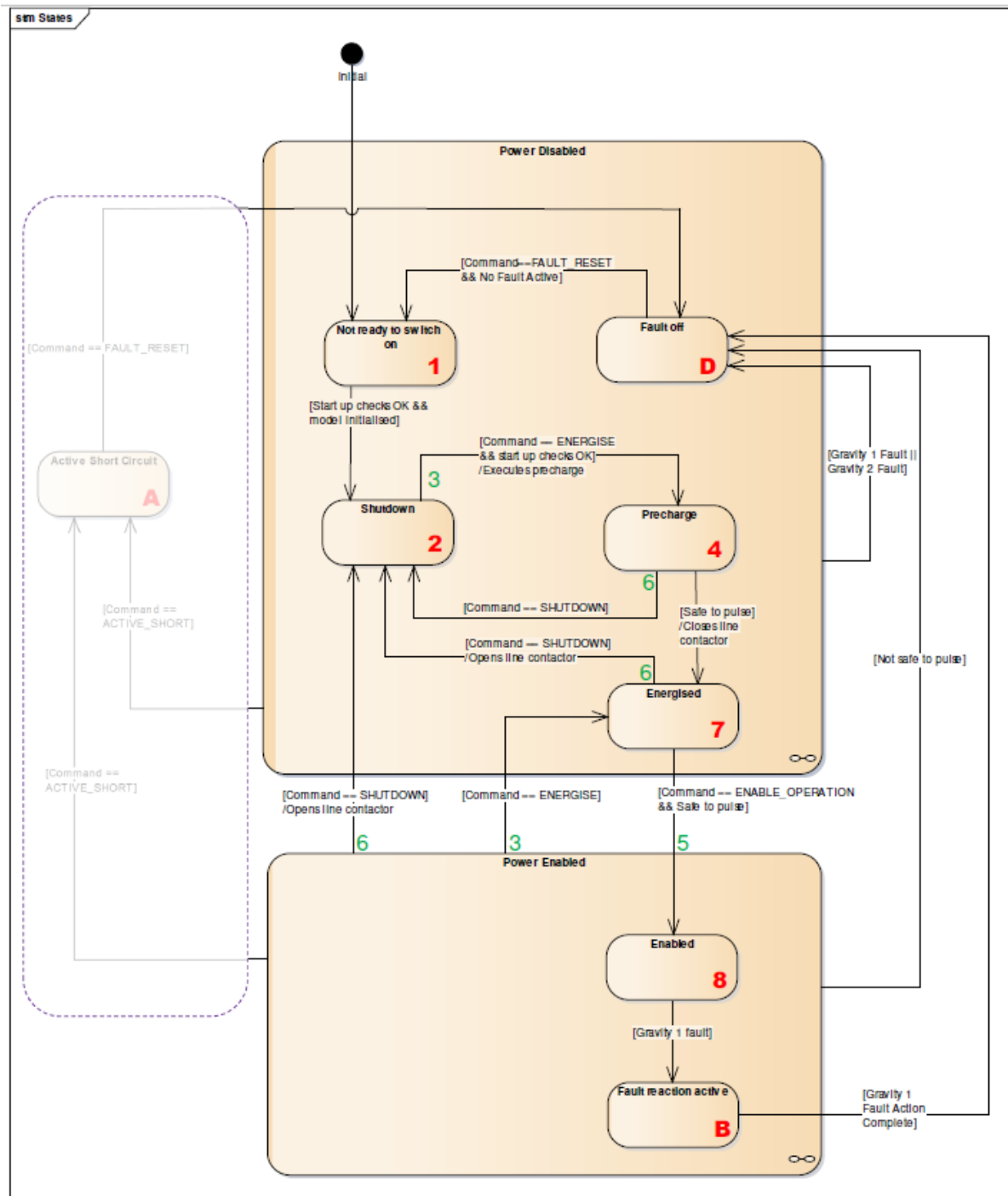


Figure 3 – H-protocol state machine (Gen4 / HVLP / Dragon8)

Note1: In Gen4 in order to move top state pre-charge the line contactor must be open. Also, if in the energised state or enabled state and the line contactor opens a fault is raised and the unit goes into state 'fault off'.

Note2: Active short is not implemented in Gen4, HVLP or Dragon8.

3.2.2 Gen5 size 9 state machine

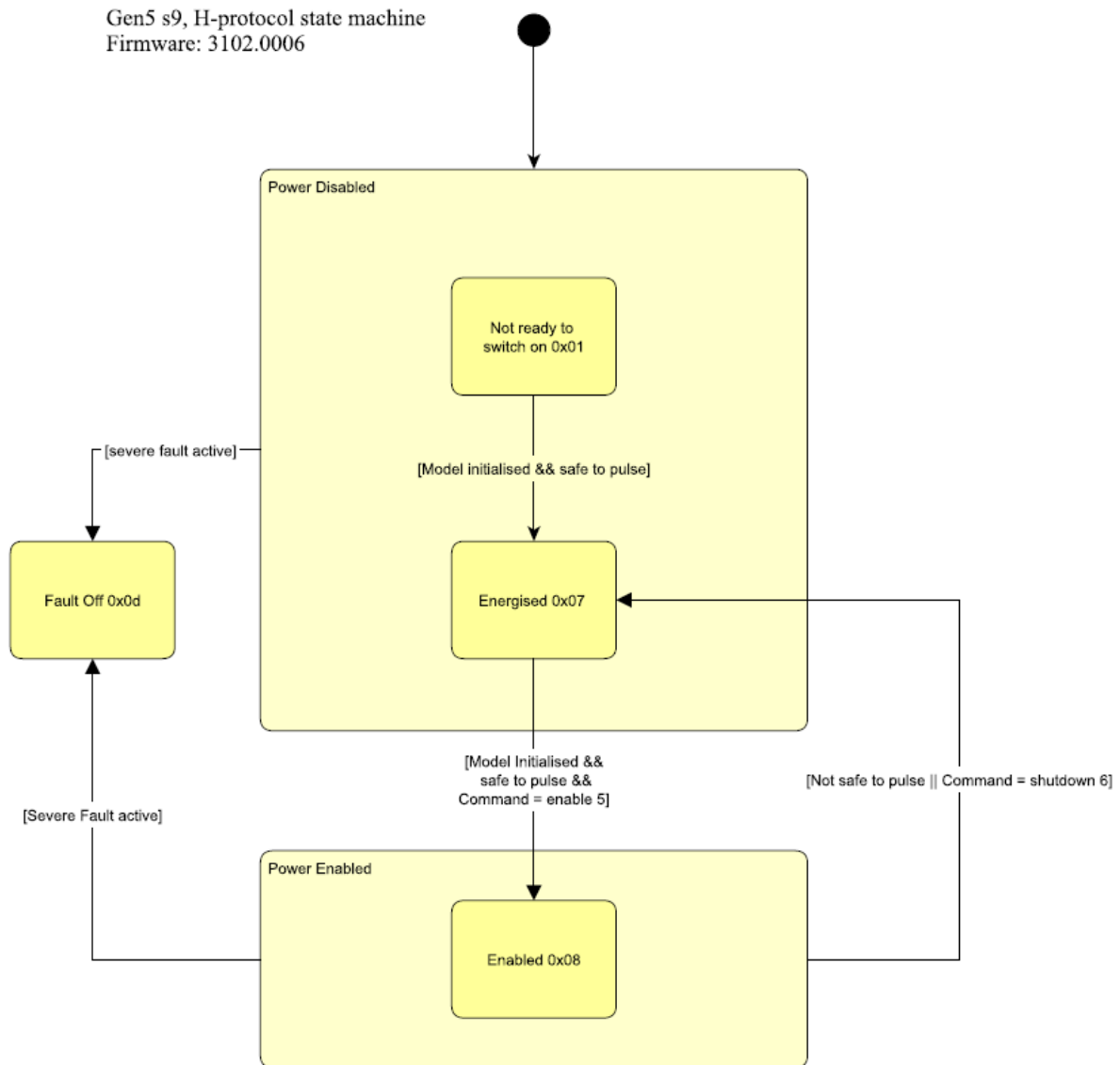


Figure 4 – H-protocol state machine (Gen5)

3.3 State Machine Implementation

The state machine runs in the node manager task that runs every 20ms for Dragon8 / HVLP / Gen5 and every 10ms for Gen4.

There are two functions that handle the super states **Power Enabled** and **Power Disabled**. These check conditions that might cause an exit from either of the super states. Note that it is possible for the state to change between calling the super state handler, and the stand-alone state handler. When

the controller is in a particular state, the functions associated with it will execute at the state machine poll rate.

Each of the states defined in the control word of message **HC1**, see section 2.3.2, have values chosen so that there are at least two bits different between any two values – in order to prevent single bit flips erroneously changing state. This is achieved by only using values whose bitwise representation contains an odd number of 1's.

3.4 System Status

Several state machine transitions are gated on various system state flags. Status flags can be grouped together into two categories, “start_up_ok” and “safe_to_pulse” – reference the state machine diagram.

For instance, in order for the state machine to transition from **Not ready to switch on** to **Shutdown**, all of the flags required for “start_up_ok” must be set. Similarly, the transition into **Energised**, and the ability to remain in the **Power Enabled** super state, all of the flags that are required for “safe_to_pulse” must be set.

The conditions differ from Gen4 to HVLP / Dragon8 / Gen5 and are listed below for clarity:

3.4.1 HVLP / Dragon8 / Gen5

Flag	Bit Mask	Required for “start_up_ok”	Required for “safe_to_pulse”
STATUS_CANOPEN_OPERATIONAL	0x0001		yes
STATUS_ENCODER_INITIALISED	0x0002		yes
STATUS_AUTOZERO_TAKEN	0x0004		yes
STATUS_THERMAL_MODEL_RUNNING	0x0008		yes
STATUS_BOTTOM_SWITCHES_OK	0x0010	yes	yes
STATUS_TOP_SWITCHES_OK	0x0020		yes
STATUS_PARAMS_RECEIVED	0x0040	yes	yes
STATUS_OVERTEMP_NOT_TRIPPED	0x0080		yes
STATUS_PF_OF_KNOWN_TYPE	0x0100	yes	yes
STATUS_INPUTS_OK	0x0200	yes	yes
STATUS_NO_SEVERE_LMC_FAULTS	0x0400		yes
STATUS_KL15_ON	0x0800	yes	yes
STATUS_HVIL_CONNECTED	0x1000		
STATUS_BATTERY_CONTACTOR_CLOSED	0x2000		
STATUS_SAFE_TO_RUN	0x4000		yes
STATUS_NOT_OVERSPEEDING	0x8000		yes

Table 30. J1939 state flags (HVLP / Dragon 8)

3.4.2 Gen4 size 8 / 10

Flag	Bit Mask	Required for "start_up_ok"	Required for "safe_to_pulse"
STATUS_BRIDGE_CHARGED	0x0001		yes
STATUS_ENCODER_INITIALISED	0x0002		yes
STATUS_AUTOZERO_TAKEN	0x0004		yes
STATUS_PULSE_INPUT_ENABLED	0x0008		yes
STATUS_BOTTOM_SWITCHES_OK	0x0010		yes
STATUS_TOP_SWITCHES_OK	0x0020		yes
STATUS_PARAMS_RECIEVED	0x0040	yes	yes
STATUS_LOGIC_VOLTAGES_ESTABLISHED	0x0080	yes	yes
STATUS_PF_OF_KNOWN_TYPE	0x0100	yes	yes
STATUS_INPUTS_OK	0x0200	yes	yes
STATUS_NO_SEVERE_FAULTS	0x0400		yes
STATUS_CAN_COMMS_OK	0x0800		
STATUS_LINE_CONTACTOR_CLOSED	0x1000		

Table 31. J1939 state flags (Gen4)

4 H-Protocol Motor Control

4.1 Start-Up Procedure

The following sequence can be performed either by a VCU, or manually when controlling the system via CANalyzer or DVT:

The expected start-up sequence is intended to be:

1. System initialises in the **Not ready to switch on** state.
2. Power up self-checks are performed. If these are good, the system transitions automatically to the **Shutdown** state the checks differ slightly between Gen4 and HVLP/Dragon8/Gen5.
3. A controlling VCU should see this state transition as a signal that it is safe to apply high voltage to the unit.
4. The VCU should then send the **ENERGISE** command via the control word. This will cause a transition into **Precharge**.
5. The controller will initiate an internal pre-charge and, when charging is complete, will transition to the **Energised** state. If the controller is in control of its own line contactor, then it will close the contactor once the pre-charge has completed. However, if the line contactor is under the control of the VCU, then the VCU should initiate its own pre-charge and close its own contactor at the same time as issuing the **Precharge** command.
6. The VCU should then send the **ENABLE OPERATION** command. This will cause a transition to **Enabled**. The bridge will be enabled, current control will start, and output torque should now be available and open to control from the VCU.
7. Should the VCU require the bridge to disable again, it can do so by sending either the **ENERGISE** command, which will disable the bridge, or the **SHUTDOWN** command, which will open the line contactor.

Note at step 4 in the above sequence, an **ENERGISE** command is required before the **ENABLE OPERATION** command. The purpose of this is so that in the event of the inverter software crashing or reset, the bridge will not instantly re-enable due to continued transmission of commands from the VCU containing the **ENABLE OPERATION** command, i.e. it forces the VCU to acknowledge the change in state and go back through its reset sequence.

Note the current state of the controller is sent to the VCU via status message **HS2** which lets the VCU know the current state of the motor controller at any point in time – reference section 0.

4.2 Control Message Timeout

Three mandatory control messages are expected to be received by the controller. These are messages **HC1**, **HC2** and **HC3**.

There is a base timeout period for each message. This is 5ms for all messages apart for **HC3**, which has a base timeout of 50ms. The actual timeout used by the software is a user configurable multiple of the base timeouts – see section 2.2.2. Typically, this would be set to a value of 3, which means that the system will enter a timeout condition if any message is absent for 15ms, or 150ms for **HC3**. However, for systems that aren't expected to receive control messages at a 5ms rate, a larger multiplier can be specified, meaning the system won't enter a timeout condition for a longer period of time. There is a 1000 msec limit to the timeout, no matter what the multiplier is set to.

When any valid message is received, the relevant variable is reset to the expected time that the next message is expected to arrive. Then, cyclically, all messages have their field decremented by the functions poll interval towards zero. Should any of them actually reach zero then flag **'h_protocol_active'** will be set to **FALSE**. The result of this can be monitored in the H-protocol status flags object 0x4655, sub-index 6, bit 11 0x0800. Also, the system will stop accepting control data, and instead revert to a set of numbers that result in zero torque output. Note that this doesn't disable the bridge, since we may need to continue with field weakening.

Some software versions will also set a warning **0x4703 J1939 Rx message timeout**

4.3 Direct Control of Id And Iq (Gen4 size 8 / 10 only)

A fourth message, named **HC4** is optionally used to control the 'd' and 'q' axis current demands. However, these demands are only ever acted upon when the control mode of operation parameter at 0x4656, 0x0103 is set to "Direct Id/Iq control".

For a full description of control message validation and synchronisation see section 0. If the **HC4** is used and control is enabled object 0x4656 sub-index 0 must be set back to 0x0000 to put the controller back into normal mode of operation.

4.4 Interface to Torque Control

The H-Torque protocol maintains two structures containing data to be transmitted and data to be received via CAN. At regular intervals, these structures are polled, and data is either loaded to or from the motor control structures and CAN transmissions are triggered.

Every 5ms the following is performed:

- Recalculate flags within the CAN module to ensure that valid CAN messages are being received, and that none of the messages have timed out.
- Performs range checking on received data, ensuring it is within range.
- The information received in control messages is copied into the motor control inputs to be acted upon by the control algorithms.
- Polling the calculation of status variables and execute an iteration of the state machine, as described in section 3.2.
- Prepares the CAN messages and queues them for transmission.

4.4.1 Torque Limits and Available Torque

A new feature provided by the H-Torque protocol over the CANopen protocol is the ability to limit the torque output when it deviates from the demand due to modification by one of the torque limits, in particular the speed loop.

In the CANopen protocol, the speed limit control loops will make use of all available torque in order to maintain a speed limit. This includes the potential to apply full braking torque to slow the motor down when the speed limit is exceeded. For traction applications this may not be desirable. For instance, it might be the case that when exceeding the speed limit, the drive torque should be reduced down to zero but, following this, there should not be an increase in braking torque. This has uses when performing operations such as engine cranking, battery protection, and avoiding potential wheel locking scenarios.

The protocol requires that torque limits are specified as a drive torque limit and a brake torque limit. Specifying torque in this way provides a convenient mechanism for a VCM to disable all drive torque. Internally, these are converted to forwards and reverse torque limits using the following mechanism:

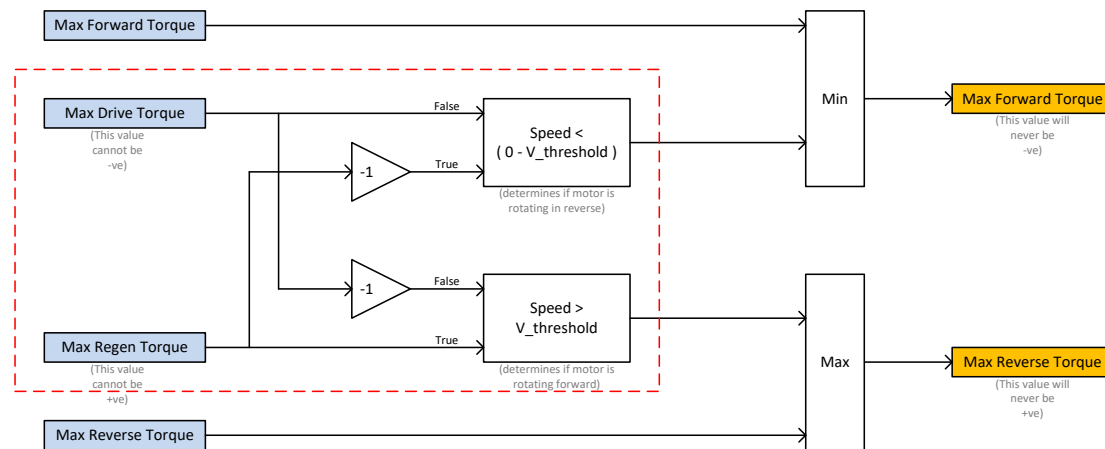


Figure 5 – motor / regen switching

The blue boxes on the left represent received torque limits. The centre portion in the red box indicates how drive and regen torques are translated to forward and reverse torque limits:

- When travelling forwards, with positive rotation, the max drive torque and max regen torque values can be assigned directly to the internal max forward and max reverse torque values, effectively forming a clamp on available torque.
- When travelling in reverse, with negative rotation, the sign of each signal is inverted, and they are switched over, such that the regen limit is in the forwards torque direction, and the drive limit is in the reverse direction.
- When stationary, defined in this case as a rotational speed $V_threshold$ (velocity threshold), the drive torque limit is applied in both directions, but has to be negated for the reverse torque limit.

$V_threshold$ (velocity threshold) can be configured in the object dictionary at 0x4623, sub-index 8.

These torque limits are processed along with other factors that can limit torque, such as battery voltage and battery current limit. These then form the limits for the rest of the system, including the speed loop.

Reporting of available torque is another addition to the H-Torque protocol over the CANopen protocol. This information is commonly required by VCUs, especially those that manage torques between different motors. Sometimes a VCU will be able to transfer torque generation to a different node, in which case it needs to know if torque is available. Or perhaps the information is used to indicate a downgraded mode to the user.

In either case, the amount of available torque can be determined by looking at the torque limits calculated by the speed limit loops. The speed loops work by calculating a torque limit for the forward and reverse direction. Since the speed loops are the last function that can limit the output torque, these limits effectively indicate the total amount of torque available.

Both the forward and reverse speed limit loops run in parallel. If there is no limit, both loops will saturate to the maximum available torque. This includes torque limitations brought about because of other factors, such as low battery or power limit maps, and also the torque limit messages. When

speed limiting, the limits are modified further. It may be that the torque output is now running on the limit of the speed loop.

The output for the forward speed limit loop becomes the available forward torque. Similarly, available reverse torque is set to the limit calculated by the reverse speed limit loop.

In general, the forward torque limit is expected to be positive and the reverse torque limit is expected to be negative, thus specifying the bounds of torque demand that can be requested. However, it should be noted that, due to the way the speed limit works, this might not always be the case.

For instance, if the forward speed limit is exceeded, we can expect available forward torque to drop. However, if an external force is continuing to push the motor over its forward speed limit, the available forward torque will pass through zero and become negative, thereby applying braking torque. This will continue to be applied up to the braking torque limit.

A similar scenario exists for exceeding the speed limit in the reverse direction.

4.4.2 Inversion of Motor Forward Direction

Object dictionary parameter 0x4655 sub-index 1 allows the user to invert the direction of forwards rotation. By setting this to either 0x5533 or 0xCCAA, the user can define whether positive torque demands result in positive or negative q-axis current references.

The inversion is done at the top end protocol level, but both the incoming and outgoing data is inverted such that, no matter whether the inversion is active or not, a positive torque demand on an unloaded motor will result in a positive direction of rotation. **Assuming the encoder has already been correctly configured.**

Underneath, if the inversion is activated, the debug messages will show that positive torque demands are producing negative current references. However, the data in the safety critical messages (i.e. those that have counters and checksums) will have data modified showing that the positive torque is being produced.

When forwards direction inversion is active, the following happens:

- Torque demands are negated
- Forward and reverse speed limits are exchanged, and their values negated
- Forward and reverse torque limits are exchanged, and their values negated
- Measured motor torque is negated
- Measured speed is negated
- Available forward and reverse torque values are exchanged and negated

The numbers 0x5533 and 0xCCAA were chosen to represent the torque directions as they were considered unique values, with a significant difference in bit pattern. If the torque direction variable happens to be anything other than 0x5533 or 0xCCAA, all torque limits, demands, reported torques and reported speeds are forced to zero.

5 H-Protocol to TracApp & PumpApp Integration

TracApp and PumpApp is supported in both Gen4 and HVLP/Dragon8 products – however only in the HVLP & Dragon8 product is H-Protocol generated from the I/O in order to drive the motor control – reference the following document that describes the I/O mapping required to generate the H-Protocol in relevant products:

AppNote – HVLP/D8 Mapping J1939 Items to VehApp

6 Differences between Gen4 H-Protocol and HVLP/D8/Gen5

In HVLP/Dragon8/Gen5 the H-Protocol is implemented as a configurable J1939 peer-peer protocol where as in Gen4 H-Protocol is implemented as a J1939 broadcast protocol with fixed PGN numbers. Also, in Gen4 for the control messages the J1939 source address field in the CAN ID is actually the destination address as configured by the VCU. In Gen4 the source address (SA) for the ECU in the status and debug messages is obtained from object dictionary parameter 0x4655 sub-index 11.

The following broadcast addresses are assigned for the Gen4 implementation of the protocol:

H-Protocol Message	Allocated Broadcast PGN
HC1	65280
HC2	65281
HC3	65282
HC4	65283
HS1	65316
HS2	65317
HS3	65318
HS4	65319
HD1	65328
HD2	65329
HD3	65330
HD4	65331
HD5	65332
HD6	65333

Table 32. Gen4 PGNs

Note: The J1939 default priority for the above J1939 messages is 6 and the Data Page (DP) setting is 0. Also, in Gen4 HS5 does not exist.